AUTOM-04 LAB: Terraform for Dynatrace

> **Series:** AUTOM — Dynatrace Automation | **Notebook:** 4 of 9 (LAB) | **Created:** April 2026 | **Last Updated:** 05/11/2026

Overview

Hands-on lab for installing the Dynatrace Terraform provider, configuring authentication, creating and managing Dynatrace resources as code, importing existing configuration, and integrating into a CI/CD pipeline with GitHub Actions.

Table of Contents

Prerequisites

Requirement	Details
Completed	AUTOM-04: Terraform Provider (lecture notebook)
Dynatrace Environment	SaaS tenant (Gen3)
Terraform CLI	Version 1.0+ installed
Authentication	API Token and/or OAuth Client credentials
Permissions	Settings write, IAM admin (for Section 5)
Git	Git CLI and a GitHub account (for Section 9)

1. Install Terraform

Terraform is a single binary. Download it from terraform.io or use a package manager.

macOS (Homebrew):

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

Linux (APT — Debian/Ubuntu):

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt-get update &amp;amp;&amp;amp; sudo apt-get install terraform
```

Windows (Chocolatey):

```
choco install terraform
```

Verify the installation:

```
terraform version
# Expected output: Terraform v1.x.x
```

> **Note:** The Dynatrace Terraform provider requires Terraform 1.0 or later. Any 1.x release will work.

---

2. Configure the Dynatrace Provider

Create a project directory and a `main.tf` file:

```
mkdir dynatrace-terraform &amp;&amp; cd dynatrace-terraform
touch main.tf
```

Add the provider block to `main.tf`:

```
terraform {
  required_providers {
    dynatrace = {
      source  = "dynatrace-oss/dynatrace"
      version = "~&gt; 1.93"
    }
  }
}
```

Authentication Method 1: API Token (Simplest)

Best for getting started. The API Token covers Settings 2.0, Synthetics, and SLOs.

```
provider "dynatrace" {
  dt_env_url   = "https://&lt;env-id&gt;.live.dynatrace.com"
  dt_api_token = var.dt_api_token
}
```

Authentication Method 2: OAuth Client Credentials (Recommended for Automation)

Required for IAM resources and Gen3 platform resources (workflows, documents, segments).

```
provider "dynatrace" {
  dt_env_url       = "https://&lt;env-id&gt;.live.dynatrace.com"
  dt_client_id     = var.dt_client_id
  dt_client_secret = var.dt_client_secret
  dt_account_id    = var.dt_account_id
}
```

> **Important (v1.88.0):** OAuth-based authentication **cannot manage synthetic monitors or SLO definitions** as of provider v1.88.0. Use an API Token for those resources, or configure both tokens together.

Authentication Method 3: Environment Variables

Keep credentials out of `.tf` files entirely:

```
export DYNATRACE_ENV_URL="https://&lt;env-id&gt;.live.dynatrace.com"
export DYNATRACE_API_TOKEN="dt0c01.xxx..."
```

With environment variables set, the provider block needs no arguments:

```
provider "dynatrace" {}
```

Initialize the Provider

Run `terraform init` to download the provider plugin:

```
terraform init
# Initializing provider plugins...
# - Installing dynatrace-oss/dynatrace v1.93.x...
# Terraform has been successfully initialized!
```

---

3. Create Your First Resource — Alerting Profile

Add an alerting profile resource to `main.tf`:

```
resource "dynatrace_alerting" "production" {
  name = "Production Alerts"
  rules {
    rule {
      severity_level   = "AVAILABILITY"
      delay_in_minutes = 0
    }
    rule {
      severity_level   = "ERROR"
      delay_in_minutes = 5
    }
  }
}
```

Preview the Change

```
terraform plan
```

Expected output:

```
Terraform will perform the following actions:

  # dynatrace_alerting.production will be created
  + resource "dynatrace_alerting" "production" {
      + id   = (known after apply)
      + name = "Production Alerts"
      + rules {
          + rule {
              + severity_level   = "AVAILABILITY"
              + delay_in_minutes = 0
            }
          + rule {
              + severity_level   = "ERROR"
              + delay_in_minutes = 5
            }
        }
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

Apply the Change

```
terraform apply
# Type "yes" when prompted to confirm
```

Verify the Resource

```
terraform show
```

Confirm the alerting profile exists in your Dynatrace tenant under **Settings > Alerting > Alerting profiles**.

---

4. Create Settings 2.0 Resources

The `dynatrace_generic_setting` resource can manage any Settings 2.0 schema. This is useful when the provider does not have a dedicated resource type for a particular setting.

Example: Kubernetes Metadata Enrichment Rule

```
resource "dynatrace_generic_setting" "k8s_enrichment" {
  schema_id = "builtin:kubernetes.generic.metadata.enrichment"
  scope     = "environment"
  value = jsonencode({
    enabled = true
    rules = [{
      sourceAttribute = "namespace-label"
      sourceKey       = "team"
      targetAttribute = "dt.security_context"
    }]
  })
}
```

Key Attributes

Attribute	Description
`schema_id`	The Settings 2.0 schema identifier (e.g., `builtin:kubernetes.generic.metadata.enrichment`)
`scope`	Where the setting applies: `environment`, `HOST-xxx`, `KUBERNETES_CLUSTER-xxx`, etc.
`value`	JSON-encoded configuration object matching the schema definition

Finding Schema IDs

Navigate to **Settings** in the Dynatrace UI. The schema ID appears in the URL when you open any setting:

```
https://&lt;env-
id&gt;.apps.dynatrace.com/ui/settings/builtin:kubernetes.generic.metadata.enr
ichment

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                                                This is the schema_id
```

Alternatively, use the Settings API:

```
curl -H "Authorization: Api-Token $DT_API_TOKEN" \
  "https://&lt;env-id&gt;.live.dynatrace.com/api/v2/settings/schemas" | jq
'.items[].schemaId'
```

---

5. Create IAM Resources (OAuth Required)

IAM resources (groups, policies, bindings) require OAuth client credentials. API tokens cannot manage IAM.

Create an IAM Group

```
resource "dynatrace_iam_group" "sre_team" {
  name = "SRE Team"
}
```

Create an IAM Policy

```
resource "dynatrace_iam_policy" "sre_policy" {
  name            = "SRE Full Access"
  statement_query = "ALLOW environment:roles:viewer;"
  tags            = ["sre"]
}
```

Bind the Policy to the Group

```
resource "dynatrace_iam_policy_bindings_v2" "sre_binding" {
  group_id   = dynatrace_iam_group.sre_team.id
  account_id = var.dt_account_id
  policies   = [dynatrace_iam_policy.sre_policy.id]
}
```

Apply

```
terraform plan
terraform apply
```

> **Note:** The `account_id` variable is required for IAM policy bindings. This is your Dynatrace account UUID, found under **Account Management > Account settings**.

---

6. Import Existing Resources

If you already have Dynatrace resources configured manually, you can bring them under Terraform management without recreating them.

Step 1: Add a Resource Block

Create an empty resource block in your `.tf` file:

```
resource "dynatrace_alerting" "existing_profile" {
  # Configuration will be filled after import
}
```

Step 2: Find the Resource ID

Locate the resource ID from the Dynatrace UI or API. For alerting profiles, it appears in the URL:

```
https://&lt;env-
id&gt;.apps.dynatrace.com/ui/settings/builtin:alerting.profile/abc12345-def6-
7890-abcd-ef1234567890

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

                                                                        This
is the resource ID
```

Step 3: Import

```
terraform import dynatrace_alerting.existing_profile abc12345-def6-7890-abcd-
ef1234567890
```

Step 4: Generate the HCL

After importing, extract the configuration from state:

```
terraform show -no-color &gt; imported.tf
```

Review and clean up the generated HCL. Remove read-only attributes (like `id`) and format the code.

Common Import Targets

Resource Type	ID Source
`dynatrace_alerting`	Settings UI URL or API
`dynatrace_dashboard`	Dashboard URL (`id` parameter)
`dynatrace_slo`	SLO list page or API
`dynatrace_generic_setting`	Settings API (`objectId` field)

> **Tip:** After import, run `terraform plan` to verify the imported state matches the live configuration. Any differences indicate drift that you should reconcile in the `.tf` file.

---

7. Variables and Multi-Environment

Define Variables

Create `variables.tf` :

```
variable "dt_env_url" {
  type        = string
  description = "Dynatrace environment URL"
}

variable "dt_api_token" {
  type        = string
  sensitive   = true
  description = "Dynatrace API token"
}

variable "environment" {
  type        = string
  default     = "dev"
  description = "Target environment name (dev, staging, production)"
}
```

Supply Values with tfvars

Create `terraform.tfvars` (add to `.gitignore` — never commit credentials):

```
dt_env_url   = "https://&lt;env-id&gt;.live.dynatrace.com"
dt_api_token = "dt0c01.xxx..."
environment  = "production"
```

Approach 1: Workspace-Based Multi-Environment

Terraform workspaces maintain separate state files per environment using the same configuration:

```
# Create workspaces
terraform workspace new dev
terraform workspace new staging
terraform workspace new production

# Switch between environments
terraform workspace select production

# Use workspace name in configuration
# terraform.workspace returns the current workspace name
```

Reference the workspace in your resources:

```
resource "dynatrace_alerting" "alerts" {
  name = "${terraform.workspace}-alerts"
  # ...
}
```

Approach 2: Directory-Based Multi-Environment

For larger teams, use separate directories with shared modules:

```
dynatrace-terraform/
  modules/
    alerting/
      main.tf
      variables.tf
  environments/
    dev/
      main.tf         # calls module with dev values
      terraform.tfvars
    staging/
      main.tf
      terraform.tfvars
    production/
      main.tf
      terraform.tfvars
```

Each environment directory runs independently with its own state and variables.

---

8. State Management

Terraform state tracks the mapping between your `.tf` files and real Dynatrace resources. By default, state is stored locally in `terraform.tfstate`.

Local State (Default)

Works for single-operator setups. The state file is created automatically after `terraform apply`.

> **Warning:** Never commit `terraform.tfstate` to version control. It may contain sensitive values. Add it to `.gitignore`.

Remote State — S3 Backend

For team use, store state in a shared remote backend:

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state"
    key            = "dynatrace/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}
```

Attribute	Purpose
`bucket`	S3 bucket for state storage
`key`	Path within the bucket
`dynamodb_table`	DynamoDB table for state locking (prevents concurrent modifications)
`encrypt`	Encrypt state at rest

Remote State — Azure Blob

```
terraform {
  backend "azurerm" {
    resource_group_name  = "rg-terraform"
    storage_account_name = "tfstateaccount"
    container_name       = "tfstate"
    key                  = "dynatrace.terraform.tfstate"
  }
}
```

Remote State — Terraform Cloud

```
terraform {
  cloud {
    organization = "my-org"
    workspaces {
      name = "dynatrace-production"
    }
  }
}
```

Inspecting State

```
# List all resources in state
terraform state list

# Show details of a specific resource
terraform state show dynatrace_alerting.production

# Remove a resource from state (without destroying it)
terraform state rm dynatrace_alerting.production
```

---

9. GitHub Actions CI/CD Pipeline

Automate Terraform deployments with GitHub Actions. The workflow runs `plan` on pull requests and `apply` on merge to main.

Workflow File

Create `.github/workflows/terraform-deploy.yml`:

```
name: Deploy Dynatrace Terraform

on:
  pull_request:
    paths: ["terraform/**"]
  push:
    branches: [main]
    paths: ["terraform/**"]

jobs:
  plan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3

      - name: Terraform Init
        run: terraform init
        working-directory: terraform/

      - name: Terraform Plan
        run: terraform plan -no-color
        working-directory: terraform/
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}

  apply:
    if: github.ref == 'refs/heads/main' &amp;&amp; github.event_name ==
'push'
    needs: plan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3

      - name: Terraform Init
        run: terraform init
        working-directory: terraform/

      - name: Terraform Apply
        run: terraform apply -auto-approve
        working-directory: terraform/
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
```

GitHub Secrets

Store credentials in GitHub repository secrets (**Settings > Secrets and variables > Actions**):

Secret Name	Value
`DT_ENV_URL`	`https://<env-id>.live.dynatrace.com`
`DT_API_TOKEN`	Your Dynatrace API token

Adding Approval Gates

For production deployments, require manual approval before `terraform apply`:

1. Create a GitHub Environment named `production` under **Settings > Environments**
2. Enable **Required reviewers** and add approvers
3. Reference the environment in the `apply` job:

```
  apply:
    environment: production   # Requires approval
    needs: plan
    # ...
```

---

10. Drift Detection

Drift occurs when someone changes a Dynatrace resource manually (through the UI or API) after it was deployed via Terraform.

Detect Drift

```
# Run plan with detailed exit code
terraform plan -detailed-exitcode

# Exit codes:
#   0 = No changes (no drift)
#   1 = Error
#   2 = Changes detected (drift found)
```

Scheduled Drift Detection

Add a scheduled GitHub Actions workflow to check for drift daily:

```
name: Drift Detection

on:
  schedule:
    - cron: "0 8 * * 1-5"   # Weekdays at 8 AM UTC

jobs:
  detect-drift:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform init
        working-directory: terraform/
      - name: Check for drift
        id: drift
        run: terraform plan -detailed-exitcode
        working-directory: terraform/
        continue-on-error: true
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
      - name: Alert on drift
        if: steps.drift.outcome == 'failure'
        run: echo "Drift detected! Review terraform plan output."
```

Handling Drift

Scenario	Action
Manual change was intentional	Update `.tf` to match, or `terraform import` the new state
Manual change was accidental	Run `terraform apply` to restore the desired state
State is stale	Run `terraform refresh` to update state from the live tenant

---

11. Summary and Checklist

Deployment Checklist

- [ ] Terraform 1.0+ installed and verified
- [ ] Provider configured with correct authentication
- [ ] `terraform init` downloads the Dynatrace provider
- [ ] Resources defined in `.tf` files, not configured manually
- [ ] `terraform.tfvars` and `terraform.tfstate` in `.gitignore`
- [ ] Remote state backend configured for team use
- [ ] CI/CD pipeline runs `plan` on PRs, `apply` on merge
- [ ] Drift detection scheduled

Key Takeaways

Concept	Key Point
Provider Setup	Pin version with `~> 1.93`, configure authentication
Authentication	API Token for Synthetics/SLOs, OAuth for IAM and Gen3
Plan Before Apply	Always review `terraform plan` output before applying
State Management	Use remote backends for team collaboration
Import	Bring existing resources under Terraform control without recreation
CI/CD	Automate with GitHub Actions; plan on PR, apply on merge
Drift Detection	Schedule regular checks with `plan -detailed-exitcode`

Monaco vs Terraform — When to Use Which

Scenario	Recommended Tool
Quick config export/import	Monaco
Multi-environment promotion	Either
IAM policy management	Terraform (OAuth required)

Scenario	Recommended Tool
Multi-cloud infrastructure + Dynatrace	Terraform
Drift detection and state tracking	Terraform
Team with existing Terraform expertise	Terraform
Team new to IaC	Monaco (lower learning curve)

References

Resource	URL
Terraform Registry	registry.terraform.io/providers/dynatrace-oss/dynatrace
Provider Documentation	registry.terraform.io/providers/dynatrace-oss/dynatrace/latest/docs
Provider GitHub	github.com/dynatrace-oss/terraform-provider-dynatrace
Terraform Install	developer.hashicorp.com/terraform/install
Dynatrace IAM Docs	docs.dynatrace.com/docs/manage/identity-access-management

---

Return to AUTOM-04: Terraform Provider for conceptual coverage, or continue to AUTOM-05: Dynatrace Workflows for event-driven automation.

Community Resources

Repository	Description
terraform-provider-dynatrace	Official provider (v1.93.0, 180 releases) with export capability
dynatrace-configuration-as-code-samples	10 starter templates in `basic-templates-terraform`, plus modules, IAM, and DQL examples
terraform_modules	Reusable module pattern for synthetic monitors
terraform_dql_example	DQL as a Terraform data source for dynamic config generation

Repository	Description
terraform_team_onboarding	IAM policies, groups, and Azure Entra ID for team provisioning
iam_tf_sample	IAM policies, Grail buckets, OpenPipelines, and segments

> **Tip:** Use the provider export feature (`./terraform-provider-dynatrace -export -env <url> -token <token>`) to generate `.tf` files from an existing tenant -- the fastest path to Terraform-managed configuration.
